

git

Débogage des problèmes de connexion réseau Git à l'aide de GIT_TRACE

 **Manu S Ajith**
1 S 8 février 2017 • 2 minutes de lecture

J'ai beaucoup voyagé ce mois-ci et travaillé à distance depuis des cafés et des lieux publics pendant mon séjour en Europe. Et le plus souvent j'étais connecté à Internet via les hot spots publics ou le Wifi gratuit fourni par les cafés ou restaurants.

La qualité des connexions était vraiment bonne, je dois l'admettre, mais un problème courant que j'ai rencontré avec ces connexions était lors de l'utilisation de git.

La plupart de ces connexions présentaient certaines restrictions concernant l'utilisation de git.

Problème

Dans l'un des cas, J'ai pu pousser vers github mais je n'ai pas pu récupérer/cloner/extraire du dépôt. Comme j'ai donné la commande

```
git fetch origin
```

il reste là pour toujours et n'a pas répondu du tout.

Débogage

Ouvrons donc notre guide de dépannage et passons à la règle 1 :

- Vérifiez votre connectivité :P

```
ping google.com
PING google.com (216.58.205.238): 56 data bytes
64 bytes from 216.58.205.238: icmp_seq=0 ttl=46 time=124.577 ms
```

Ok, donc je suis connecté.!!

- Vérifiez si vous avez accès au dépôt

Puisque je suis capable de pousser vers le dépôt, je suis presque sûr d'avoir accès au dépôt, mais quand même :check:

- Vérifiez si ssh a été désactivé.

```
$ ssh git@github.com

PTY allocation request failed on channel 0
Hi manusajith! You've successfully authenticated, but GitHub does not provide shell access
Connection to github.com closed.
```

D'accord.!! donc ssh n'est certainement pas bloqué sur le routeur pour le moment.

- Remplacer `git://` le protocole par `http/https`

J'étais presque sûr que ce n'était pas le problème, mais essayons quand même. !!

- Essayez de changer votre DNS en `8.8.8.8`

Je l'avais déjà eu :P

Comme tout ce qui précède a échoué, j'ai réalisé qu'il fallait un débogage un peu plus sérieux.

Mes premières pensées ont été de dépanner les connexions réseau et de consigner les tentatives de connexion à GitHub de manière plus détaillée afin de pouvoir les déboguer en profondeur.

GIT_TRACE

Bienvenue à `GIT_TRACE` . Cette option de configuration nous donne une trace plus détaillée des connexions réseau git et de toutes les commandes internes qu'elles traversent.

Cette variable d'environnement peut accepter les valeurs suivantes :

- 1, 2 ou vrai

Si cette variable est définie sur `1` , `2` ou `true` (la comparaison n'est pas sensible à la casse), git imprimera des messages trace: sur stderr indiquant l'expansion des alias, l'exécution des commandes intégrées et l'exécution des commandes externes.

- supérieur à 1 et inférieur à 10

Si cette variable est définie sur une valeur entière supérieure à 1 et inférieure à 10 (strictement), alors git interprétera cette valeur comme un descripteur de fichier ouvert et tentera d'écrire les messages de trace dans ce descripteur de fichier.

- chemin absolu

Alternativement, si cette variable est définie sur un chemin absolu (commençant par un caractère /), git l'interprétera comme un chemin de fichier et tentera d'y écrire les messages de trace.

```
$ GIT_TRACE=1 git fetch origin

13:44:47.299097 git.c:350          trace: built-in: git 'fetch' 'origin'
13:44:47.403611 run-command.c:336 trace: run_command: 'ssh' 'git@github.com' 'git
```

Avec la trace plus approfondie, j'ai pu comprendre qu'il se bloquait pour toujours lors de la tentative de téléchargement du pack.

Puisque je sais où se trouvait exactement le problème, je suis maintenant en mesure de trouver une solution au même problème, des recherches et un débogage plus approfondis m'ont donné des indices sur la raison pour laquelle il est resté bloqué là pour toujours : il s'agissait d'un type de problème de service avec la gestion des paquets TCP sur mon routeur. , ce qui ne pouvait malheureusement pas faire grand-chose.

Inscrivez-vous pour en savoir plus.

Entrer votre Email

S'abonner

Écrire du code Rust NIF pour convertir le code de cryptage d'Elixir en Rust

Dans cet article, nous verrons ce qu'est un NIF et comment écrire du code NIF sécurisé à l'aide de Rust et Rustler.

27 décembre 2022 · 3 minutes de lecture

Ajoutez le suivi de la latence aux applications Phoenix Live View.

Ajouter le suivi de la latence aux applications Phoenix Live View

16 octobre 2022 · 1 minute de lecture

Exécuter des tâches cron planifiées dans Rust à l'aide de Tokio

Exécuter des tâches périodiques/planifiées dans Rust à l'aide de tokio et async/await

15 octobre 2022